Tema: Módulo – Python, Tipos de Datos – Parte 1

Comentarios

```
# Comentario
```

¿Qué son?

Son líneas que el intérprete de Python ignora porque son precedidas de un #.

¿Para qué sirven?

Para documentar el código y no ejecutarlo cuando corre.

Tipos de comentarios

Hay dos tipos de comentarios:

Monocomentario que ocupa una línea:

```
# Esto es un comentario de una línea
print("Hola mundo")  # Comentario al final
```

Multicomentario que ocupa varias líneas:

```
"""
Esto es un comentario
que ocupa varias líneas
"""
print("Hola mundo")
```

Literales, variables y tipos de datos básicos

Los literales son valores que se escriben directamente en el código fuente (no son variables).

Los tipos de datos básicos son los que Python usa por defecto para representar números, texto y valores lógicos.

Literales

- Literales: valores escritos de forma directa (números, texto, booleanos).
- Variables: nombres que apuntan a objetos en memoria.
- Tipos de datos básicos: int, float, str, bool, NoneType.

Literales simples

- Enteros
- Flotantes
- Booleanos

```
42          # Entero decimal
3.14        # Flotante
True        # Booleano verdadero
False       # Booleano falso
```

Literales compuestos

```
"hola"      # Cadena
[1, 2, 3]   # Lista
(1, 2, 3)   # Tupla
{1, 2, 3}   # Conjunto
{"a": 1}    # Diccionario
```

```
# Enteros
42
-7
0
# Flotantes
3.14
-0.5
2.0
# Booleanos
True
False
# Cadenas
"hola"
'mundo'
"""texto
multilínea"""
# Listas
[1, 2, 3]
["a", "b", "c"]
[True, False]
# Tuplas
(1, 2, 3)
("x", "y")
# Conjuntos
{1, 2, 3}
{"a", "b"}
# Diccionarios
{"a": 1, "b": 2}
{"nombre": "Ana", "edad": 30}
# None
None
```

Variables

- Definición:
- Nombre: identificador que permite acceder a un valor en memoria.
- Asignación: operador = que vincula un nombre con un valor.
- Reasignación: puede cambiar el valor o incluso el tipo del nombre.
- Tipado dinámico: no hace falta declarar el tipo de una variable.

```
x = 10
x = 3.14
x = True
x = None
```

```
# Asignación simple
x = 10
# Reasignación
x = 3.14
# Cambio de tipo
x = "hola"
# Asignación múltiple
a, b = 1, 2
# Asignación de varios valores a una misma variable
x = y = 0
# Intercambio de valores
a, b = b, a
```

Identificadores

- Reglas:
- Letras, dígitos y guiones bajos, no empiezan con dígito.
- Distinguen mayúsculas y minúsculas.
- No pueden ser palabras reservadas (ej. for, if, class).
- Convenciones:
- snake_case para variables y funciones.
- UPPER_CASE_WITH_UNDERSCORE para constantes.
- PascalCase para clases.
- Evitar nombres de clases que sobrescriben nombres de clases de la biblioteca estándar.

```
# Válidos
nombre_usuario = "Ana"
edad = 30
PI = 3.14
class MiClase:
    pass
# Inválidos
2nombre = "Ana"   # empieza con dígito
mi-variable = 5   # guión medio
class = "x"       # palabra reservada
```

Literales

```
# Enteros
42
-7
0
# Flotantes
3.14
-0.5
2.0
# Booleanos
True
False
# Cadenas
"hola"
'mundo'
"""texto
multilínea"""
# None
None
```

Reserva y los nombres de los variables

- Es una palabra reservada que no puede usarse como nombre de variable, función o clase.
- Ejemplos: if, for, while, class, def, return, True, False, None, and, or, not, in, is, try, except, finally, raise, lambda, yield, pass, break, continue.
- Si intentas usarla, Python da SyntaxError.
- Ejemplo: class = "x" → error.
- Convención: si quieres usar un nombre parecido, añade un guión bajo al final: class_, type_, id_.

```
class = "x"  # SyntaxError
class_ = "x"  # OK
```

Asignación múltiple de variables

```
a, b = 1, 2
x = y = 0
a, b = b, a
```

```
# Asignación múltiple
a, b = 1, 2
# Asignación de varios valores a una misma variable
x = y = 0
# Intercambio de valores
a, b = b, a
```

Tipos de datos básicos

int

- Enteros sin límite. Tipo: <class 'int'>

```
42
-7
0
1000000
```

float

- Flotantes. Tipo: <class 'float'>

```
3.14
-0.5
2.0
1.5e3
```

bool

- True / False también son subclases de int (por eso 0 y False se comportan igual).

```
True
False
True + True  # 2
True * 5     # 5
```

NoneType

- None. Tipo: <class 'type(None)'>

```
None
```

Numeros

- Enteros: int, sin límite de tamaño.
- Flotantes: float, IEEE 754 doble precisión (~15–17 dígitos significativos).
- Notación científica: 1.5e3 → 1500.0.
- Booleanos: bool, subclase de int.
- True → 1, False → 0.
- Permiten operaciones aritméticas: True + True → 2.
- Nulo: NoneType.
- Único valor: None.
- Se usa para indicar ausencia de valor o resultado vacío.

```
# Enteros
42
-7
0
1000000
# Flotantes
3.14
-0.5
2.0
1.5e3
# Booleanos
True
False
True + True  # 2
True * 5     # 5
# Nulo
None
```

Strings

- Cadenas de texto.
- Se pueden usar comillas simples o dobles.
- Se pueden usar comillas triples para cadenas multilínea.
- Ejemplo:

```
"hola"
'mundo'
"""texto
multilínea"""
```

Conversión entre tipos

- Cada clase permite una forma de convertir otros tipos.
- Ejemplo: int("42") → 42, int(3.14) → 3, int(True) → 1.
- Advertencia: int("3.14") → ValueError.

```
int("42")   # 42
int("-7")   # -7
int("0")    # 0
int(3.14)   # 3
int(True)   # 1
int(False)  # 0
int("3.14")  # ValueError
int("hola")  # ValueError
```

Operadores

- Aritmeticos/numericos
- Comparativos: strings, números, secuencias
- Lógicos: and, or, not

Operadores aritmeticos

Operador	Nombre	Ejemplo
+	Suma	5 + 2 → 7
-	Resta	5 - 2 → 3
*	Multiplicación	5 * 2 → 10
/	División real	5 / 2 → 2.5
//	División entera	5 // 2 → 2
%	Módulo (resto)	5 % 2 → 1
**	Potencia	5 ** 2 → 25

```
5 + 2   # 7
5 - 2   # 3
5 * 2   # 10
5 / 2   # 2.5
5 // 2  # 2
5 % 2   # 1
5 ** 2  # 25
```

Operadores de comparación

Operador	Nombre	Ejemplo
==	Igual a	5 == 5 → True
!=	Distinto de	5 != 3 → True
>	Mayor que	5 > 3 → True
<	Menor que	3 < 5 → True
>=	Mayor o igual que	5 >= 5 → True
<=	Menor o igual que	3 <= 5 → True

```
5 == 5  # True
5 != 3  # True
5 > 3   # True
3 < 5   # True
5 >= 5  # True
3 <= 5  # True
```

Operadores lógicos

Operador	Nombre	Ejemplo
and	Y lógico	True and False → False
or	O lógico	True or False → True
not	Negación	not True → False

```
True and False  # False
True or False   # True
not True        # False
```

Operadores Binarios / Binarios

Operador	Nombre
&	AND binario
|	OR binario
^	XOR binario
~	NOT binario
<<	Desplazamiento a la izquierda
>>	Desplazamiento a la derecha

```
5 & 3   # 1
5 | 3   # 7
5 ^ 3   # 6
~5      # -6
5 << 1  # 10
5 >> 1  # 2
```

Operadores de Asignación

Operador	Nombre
=	Asignación
+=	Suma y asignación
-=	Resta y asignación
*=	Multiplicación y asignación
/=	División real y asignación
//=	División entera y asignación
%=	Módulo y asignación
**=	Potencia y asignación
&=	AND binario y asignación
|=	OR binario y asignación
^=	XOR binario y asignación
>>=	Desplazamiento a la derecha y asignación
<<=	Desplazamiento a la izquierda y asignación

```
x = 5
x += 2  # x = x + 2
x -= 1  # x = x - 1
x *= 3  # x = x * 3
x /= 2  # x = x / 2
x //= 2 # x = x // 2
x %= 5  # x = x % 5
x **= 2 # x = x ** 2
x &= 3  # x = x & 3
x |= 4  # x = x | 4
x ^= 1  # x = x ^ 1
x >>= 1 # x = x >> 1
x <<= 2 # x = x << 2
```

Operadores de Identidad

Operador	Nombre
is	Son el mismo objeto en memoria (misma dirección)
is not	Son distintos objetos (aunque sean iguales)

```
a is b
a is None
x is True
```

Operadores de Pertenencia

Operador	Nombre
in	Pertenece a (secuencia o colección)
not in	No pertenece a (secuencia o colección)

```
"a" in "hola"
"x" not in "hola"
3 in [1, 2, 3]
5 not in (1, 2, 3)
```

Estado de valores

- Truthy: valores que se evalúan como True.
- Falsy: valores que se evalúan como False.

```
True
False
None
0
0.0
""
[]
()
{}
set()
```

Operadores lógicos: and, or, not

Operadores de comparación: ==, !=, <, >, <=, >=, is, is not, in, not in

Operadores aritméticos: +, -, *, /, //, %, **

Operadores binarios: &, |, ^, ~, <<, >>

Operadores de asignación: =, +=, -=, *=, /=, //=, %=, **=, &=, |=, ^=, >>=, <<=

Operadores de identidad: is, is not

Operadores de pertenencia: in, not in

Estado de valores: Truthy: True, 1, "texto", [1], {1}, (1,) Falsy: False, 0, 0.0, "", [], (), {}, set(), None

Operadores de comparación: ==, !=, <, >, <=, >=, is, is not, in, not in

Operadores aritméticos: +, -, *, /, //, %, **

Operadores binarios: &, |, ^, ~, <<, >>

Operadores de asignación: =, +=, -=, *=, /=, //=, %=, **=, &=, |=, ^=, >>=, <<=

Operadores de identidad: is, is not

Operadores de pertenencia: in, not in

```
# Operadores aritméticos
5 + 2   # 7
5 - 2   # 3
5 * 2   # 10
5 / 2   # 2.5
5 // 2  # 2
5 % 2   # 1
5 ** 2  # 25
# Operadores de comparación
5 == 5  # True
5 != 3  # True
5 > 3   # True
3 < 5   # True
5 >= 5  # True
3 <= 5  # True
# Operadores lógicos
True and False  # False
True or False   # True
not True        # False
# Operadores binarios / binarios
5 & 3   # 1
5 | 3   # 7
5 ^ 3   # 6
~5      # -6
5 << 1  # 10
5 >> 1  # 2
# Operadores de asignación
x = 5
x += 2  # x = x + 2
x -= 1  # x = x - 1
x *= 3  # x = x * 3
x /= 2  # x = x / 2
x //= 2 # x = x // 2
x %= 5  # x = x % 5
x **= 2 # x = x ** 2
x &= 3  # x = x & 3
x |= 4  # x = x | 4
x ^= 1  # x = x ^ 1
x >>= 1 # x = x >> 1
x <<= 2 # x = x << 2
# Operadores de identidad
a is b
a is None
x is True
# Operadores de pertenencia
"a" in "hola"
"x" not in "hola"
3 in [1, 2, 3]
5 not in (1, 2, 3)
```

3 Tipos de datos compuestos (colecciones)

Listas

- Secuencia de datos.
- Mutable.
- Ordenada.
- Indexada.
- Permite duplicados.
- Soporta slicing.
- Soporta anidamiento.
- Ejemplo: [1, 2, 3], ["a", "b", "c"], [True, False], [1, "hola", True].

```
# Listas
[1, 2, 3]
["a", "b", "c"]
[True, False]
[1, "hola", True]
# Acceso
lista[0]
lista[-1]
# Slicing
lista[1:3]
lista[:2]
lista[2:]
lista[::2]
# Anidamiento
[[1, 2], [3, 4]]
# Mutabilidad
lista[0] = 10
lista.append(4)
lista.pop()
lista.remove(2)
lista.insert(1, 5)
lista.sort()
lista.reverse()
# Operadores
[1, 2] + [3, 4]  # [1, 2, 3, 4]
[0] * 3          # [0, 0, 0]
len(lista)
sum(lista)
min(lista)
max(lista)
# Comprensiones
[x for x in range(10)]
[x for x in range(10) if x % 2 == 0]
[x * 2 for x in lista]
# Iteración
for x in lista:
    print(x)
for i, x in enumerate(lista):
    print(i, x)
# Copia
lista.copy()
lista[:]
# Comparación
[1, 2] == [1, 2]  # True
[1, 2] == [2, 1]  # False
# Pertenencia
1 in [1, 2, 3]
5 not in [1, 2, 3]
# Unpacking
a, b, c = [1, 2, 3]
a, *b = [1, 2, 3]
*a, b = [1, 2, 3]
# Truthy / Falsy
[1]  # Truthy
[]   # Falsy
# Conversión
list("hola")  # ["h", "o", "l", "a"]
list(range(5))  # [0, 1, 2, 3, 4]
# Ejemplos reales
# Filtrar pares
pares = [x for x in range(10) if x % 2 == 0]
# Duplicar valores
dobles = [x * 2 for x in lista]
# Contar elementos
len(lista)
# Suma total
sum(lista)
# Buscar el mayor
max(lista)
# Ordenar sin modificar
sorted(lista)
# Invertir sin modificar
lista[::-1]
```

Operadores de Pertenencia

```
# Operadores de pertenencia
"a" in "hola"
"x" not in "hola"
3 in [1, 2, 3]
5 not in (1, 2, 3)
```

Tuplas

- Secuencia de datos.
- Inmutable.
- Ordenada.
- Indexada.
- Permite duplicados.
- Soporta slicing.
- Soporta anidamiento.
- Ejemplo: (1, 2, 3), ("a", "b", "c"), (True, False), (1, "hola", True).

```
# Tuplas
(1, 2, 3)
("a", "b", "c")
(True, False)
(1, "hola", True)
# Acceso
tupla[0]
tupla[-1]
# Slicing
tupla[1:3]
tupla[:2]
tupla[2:]
tupla[::2]
# Anidamiento
((1, 2), (3, 4))
# Inmutabilidad
tupla[0] = 10  # TypeError
# Operadores
(1, 2) + (3, 4)  # (1, 2, 3, 4)
(0) * 3          # (0, 0, 0)
len(tupla)
sum(tupla)
min(tupla)
max(tupla)
# Comprensiones
tuple(x for x in range(10))
tuple(x for x in range(10) if x % 2 == 0)
tuple(x * 2 for x in tupla)
# Iteración
for x in tupla:
    print(x)
for i, x in enumerate(tupla):
    print(i, x)
# Copia
tupla.copy()
tupla[:]
# Comparación
(1, 2) == (1, 2)  # True
(1, 2) == (2, 1)  # False
# Pertenencia
1 in (1, 2, 3)
5 not in (1, 2, 3)
# Unpacking
a, b, c = (1, 2, 3)
a, *b = (1, 2, 3)
*a, b = (1, 2, 3)
# Truthy / Falsy
(1)  # Truthy
()   # Falsy
# Conversión
tuple("hola")  # ("h", "o", "l", "a")
tuple(range(5))  # (0, 1, 2, 3, 4)
# Ejemplos reales
# Filtrar pares
pares = tuple(x for x in range(10) if x % 2 == 0)
# Duplicar valores
dobles = tuple(x * 2 for x in tupla)
# Contar elementos
len(tupla)
# Suma total
sum(tupla)
# Buscar el mayor
max(tupla)
# Ordenar sin modificar
sorted(tupla)
# Invertir sin modificar
tupla[::-1]
```

Operadores de Pertenencia

```
# Operadores de pertenencia
"a" in "hola"
"x" not in "hola"
3 in [1, 2, 3]
5 not in (1, 2, 3)
```

Conjuntos

- Secuencia de datos.
- Mutable.
- Desordenado.
- No indexado.
- No permite duplicados.
- No soporta slicing.
- No soporta anidamiento.
- Ejemplo: {1, 2, 3}, {"a", "b", "c"}, {True, False}, {1, "hola", True}.

```
# Conjuntos
{1, 2, 3}
{"a", "b", "c"}
{True, False}
{1, "hola", True}
# Acceso
# No indexado
# Mutabilidad
conjunto.add(4)
conjunto.remove(2)
conjunto.discard(2)
conjunto.pop()
conjunto.clear()
# Operadores
{1, 2} | {3, 4}  # {1, 2, 3, 4}
{1, 2} & {2, 3}  # {2}
{1, 2} - {2, 3}  # {1}
{1, 2} ^ {2, 3}  # {1, 3}
len(conjunto)
sum(conjunto)
min(conjunto)
max(conjunto)
# Comprensiones
{x for x in range(10)}
{x for x in range(10) if x % 2 == 0}
{x * 2 for x in conjunto}
# Iteración
for x in conjunto:
    print(x)
# Copia
conjunto.copy()
# Comparación
{1, 2} == {1, 2}  # True
{1, 2} == {2, 1}  # True
# Pertenencia
1 in {1, 2, 3}
5 not in {1, 2, 3}
# Unpacking
# No soporta unpacking
# Truthy / Falsy
{1}  # Truthy
set()  # Falsy
# Conversión
set("hola")  # {"h", "o", "l", "a"}
set(range(5))  # {0, 1, 2, 3, 4}
# Ejemplos reales
# Eliminar duplicados
sin_duplicados = set(lista)
# Contar elementos
len(conjunto)
# Suma total
sum(conjunto)
# Buscar el mayor
max(conjunto)
# Ordenar sin modificar
sorted(conjunto)
```

Operadores de Pertenencia

```
# Operadores de pertenencia
"a" in "hola"
"x" not in "hola"
3 in {1, 2, 3}
5 not in {1, 2, 3}
```

Diccionarios

- Secuencia de datos.
- Mutable.
- Ordenado.
- Indexado.
- No permite duplicados.
- No soporta slicing.
- No soporta anidamiento.
- Ejemplo: {"a": 1}, {"nombre": "Ana", "edad": 30}.

```
# Diccionarios
{"a": 1}
{"nombre": "Ana", "edad": 30}
# Acceso
diccionario["a"]
diccionario.get("a")
diccionario.get("a", 0)
# Mutabilidad
diccionario["a"] = 10
diccionario.update({"b": 2})
diccionario.pop("a")
diccionario.popitem()
diccionario.clear
# Operadores
{"a": 1} | {"b": 2}  # {"a": 1, "b": 2}
len(diccionario)
sum(diccionario.values())
min(diccionario.values())
max(diccionario.values())
# Comprensiones
{x: x * 2 for x in range(10)}
{x: x * 2 for x in range(10) if x % 2 == 0}
# Iteración
for k in diccionario:
    print(k)
for k, v in diccionario.items():
    print(k, v)
# Copia
diccionario.copy()
# Comparación
{"a": 1} == {"a": 1}  # True
{"a": 1} == {"a": 2}  # False
# Pertenencia
"a" in {"a": 1}
"b" not in {"a": 1}
# Unpacking
# No soporta unpacking
# Truthy / Falsy
{"a": 1}  # Truthy
{}  # Falsy
# Conversión
dict(zip("abc", [1, 2, 3]))  # {"a": 1, "b": 2, "c": 3}
# Ejemplos reales
# Contar elementos
len(diccionario)
# Suma total
sum(diccionario.values())
# Buscar el mayor
max(diccionario.values())
# Ordenar sin modificar
sorted(diccionario.items())
```

Operadores de Pertenencia

```
# Operadores de pertenencia
"a" in {"a": 1}
"b" not in {"a": 1}
```

Strings

- Secuencia de datos.
- Inmutable.
- Ordenada.
- Indexada.
- Permite duplicados.
- Soporta slicing.
- No soporta anidamiento.
- Ejemplo: "hola", "mundo", "texto multilínea".

```
# Strings
"hola"
"mundo"
"""texto
multilínea"""
# Acceso
cadena[0]
cadena[-1]
# Slicing
cadena[1:3]
cadena[:2]
cadena[2:]
cadena[::2]
# Inmutabilidad
cadena[0] = "x"  # TypeError
# Operadores
"hola" + "mundo"  # "holamundo"
"x" * 3           # "xxx"
len(cadena)
# Comprensiones
"".join(x for x in range(10))
"".join(x for x in range(10) if x % 2 == 0)
"".join(x * 2 for x in cadena)
# Iteración
for x in cadena:
    print(x)
for i, x in enumerate(cadena):
    print(i, x)
# Copia
cadena.copy()
cadena[:]
# Comparación
"hola" == "hola"  # True
"hola" == "mundo"  # False
# Pertenencia
"a" in "hola"
"x" not in "hola"
# Unpacking
# No soporta unpacking
# Truthy / Falsy
"hola"  # Truthy
""  # Falsy
# Conversión
str(42)  # "42"
str(True)  # "True"
# Ejemplos reales
# Contar elementos
len(cadena)
# Buscar el mayor
max(cadena)
# Ordenar sin modificar
sorted(cadena)
# Invertir sin modificar
cadena[::-1]
```

Operadores de Pertenencia

```
# Operadores de pertenencia
"a" in "hola"
"x" not in "hola"
```

Operadores de Pertenencia

```
# Operadores de pertenencia
"a" in "hola"
"x" not in "hola"
3 in [1, 2, 3]
5 not in (1, 2, 3)
"a" in {"a": 1}
"b" not in {"a": 1}
"a" in "hola"
"x" not in "hola"
```